PROJECT PACKAGE / VERSION 1.0

RV32I

Single-Cycle Processor

RTL, simulation and verification

A modular implementation with checked instruction traces, byte-accurate memory operations and a reproducible Vivado workflow.

37	52	12,604
Datapath instructions	Test programs	Retirement checks

Included

Nine Verilog RTL modules; six self-checking SystemVerilog testbenches; directed and seeded programs; an independent Python architectural model; editable block diagrams; Vivado scripts; actual verification logs.

Evidence status

Icarus simulation, GNU assembler comparison and Yosys generic synthesis were executed successfully. Vivado/XSim and FPGA hardware execution remain for local reproduction.

Prepared 09 October 2026. New implementation inspired by the linked learning repository; attribution and primary references appear on page 16.

02 / DESIGN AND VERIFICATION

Scope and architecture

The design follows the RV32I base integer instruction behavior in a deliberately small execution environment.

Family	Instructions	Count
Register arithmetic	ADD, SUB, SLL, SLT, SLTU, XOR, SRL, SRA, OR, AND	10
Immediate arithmetic	ADDI, SLTI, SLTIU, XORI, ORI, ANDI, SLLI, SRLI, SRAI	9
Loads	LB, LH, LW, LBU, LHU	5
Stores	SB, SH, SW	3
Conditional branches	BEQ, BNE, BLT, BGE, BLTU, BGEU	6
Upper immediates	LUI, AUIPC	2
Unconditional jumps	JAL, JALR	2

Three additional instruction groups

FENCE is a no-op because the supplied memories are strongly ordered and there is one hart. ECALL and EBREAK stop execution with metadata. These bring the exercised mnemonic bins to 40; they do not add datapath arithmetic operations.

Machine model

32 registers, 32-bit addresses and data, little-endian memory and fixed 32-bit instructions. Instructions and data have separate 4 KiB spaces in the simulation wrapper. One instruction can complete on each clock edge after reset, provided the combinational memory contract is met.

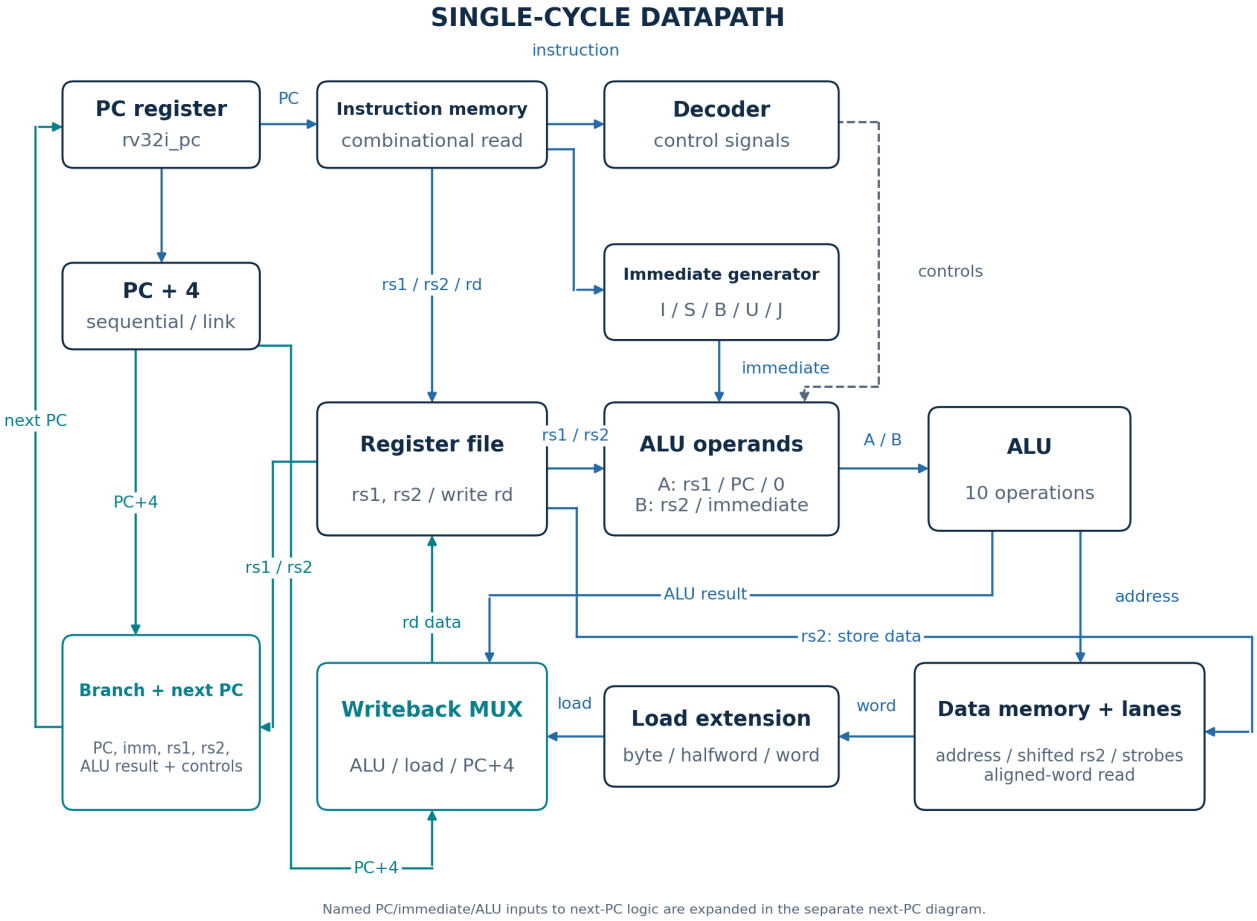
Boundary of the project

There is no pipeline, cache, memory handshake, privileged state, CSR file, interrupt path, M/C/A extension or FENCE.I. Traps halt rather than enter a software handler. This is an educational implementation, not a certified RISC-V processor or an OS-ready system.

03 / DESIGN AND VERIFICATION

The datapath

Follow the blue data paths first. The teal paths carry writeback and control flow; the decoder supplies control selections.



Named PC/immediate/ALU inputs to next-PC logic are expanded in the separate next-PC diagram.

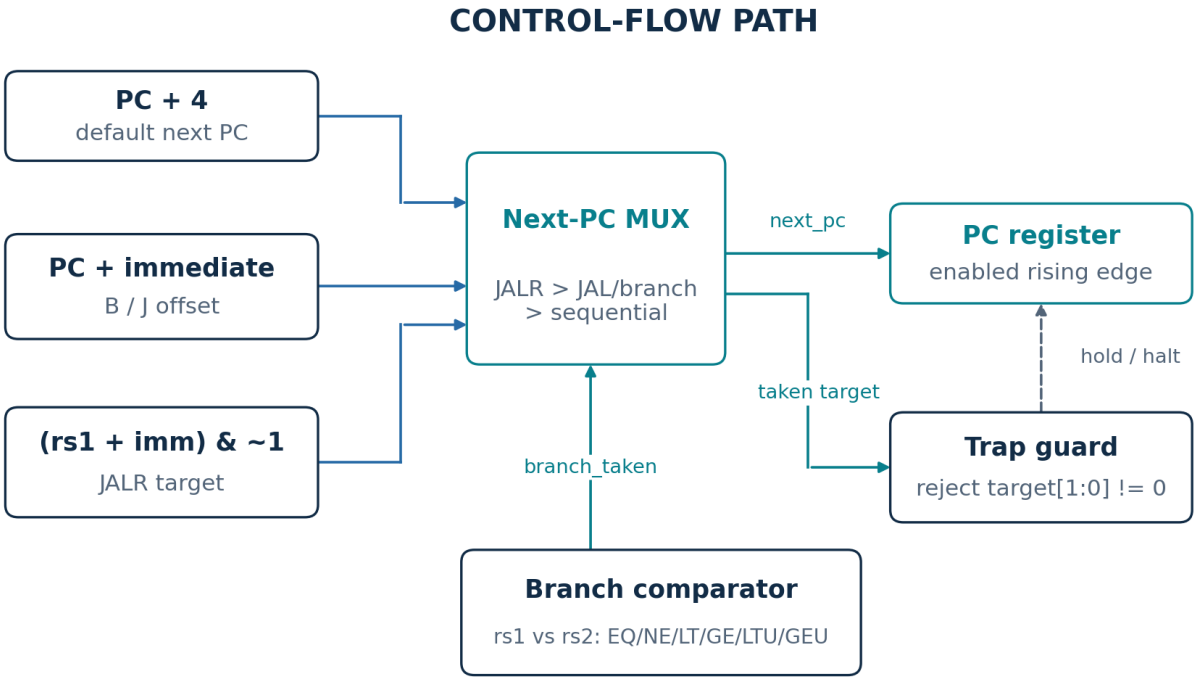
Step	What happens
Fetch	PC selects a word in instruction memory.
Decode / read	Instruction fields select two GPRs; the decoder and immediate generator interpret the instruction.
Execute / memory	Operand MUXes feed the ALU; branches compare GPRs; loads/stores use the ALU address.
Complete	Writeback selects ALU, load data or PC+4. At the edge, state changes or a trap is latched.

The next-PC block includes comparison and target selection; its named PC/immediate/ALU inputs are expanded on the next page. Fault checking gates PC, register and store updates.

04 / DESIGN AND VERIFICATION

PC, branches and jumps

The value written to a register and the next instruction address are separate choices.



Instruction	Next PC	Register writeback
Normal / untaken branch	PC + 4	Instruction-dependent
Taken branch	PC + B immediate	None
JAL	PC + J immediate	rd = PC + 4
JALR	(rs1 + I immediate) & ~1	rd = PC + 4

Why PC+4 is a third writeback input

A jump stores the address of the following instruction as a return link. The improved core therefore has three writeback sources: ALU result, extended load data and PC+4. JAL with rd=x0 performs a jump without changing a register.

JALR clears bit 0, then the core checks four-byte instruction alignment because compressed instructions are not supported. Only a taken branch's target is checked for misalignment.

05 / DESIGN AND VERIFICATION

Single-cycle timing

Combinational results become available during a cycle. Register-file storage changes at the next rising edge.

Moment	Processor activity
Rising edge N	PC and any previous instruction's architectural updates take effect.
Between N and N+1	Fetch, decode, GPR read, ALU, optional data read and writeback selection settle.
Rising edge N+1	Write the destination GPR, commit store bytes and update PC; register the retirement event.

ADD example

```
add x3, x1, x2
ALU A = x1, ALU B = x2
writeback_data = ALU result
x3 changes on the next rising edge, not immediately after the ALU.
```

LW example

```
lw x3, 8(x1)
address = x1 + 8
combinational data-memory word -> load extension -> writeback
x3 changes on the same completion edge as the PC update.
```

Memory choice matters

A synchronous-read RAM would return the load word after the edge at which this core expects to write the register. The supplied memory uses asynchronous reads and rising-edge stores, so the LW path remains single-cycle.

On an FPGA, this implies distributed memory or registers. Replacing it with block RAM requires a multi-cycle or pipelined design with suitable control. The longest path often includes instruction fetch, decode, ALU and load read; a slower clock may be necessary.

Reading a waveform

After an edge, trace_pc describes the instruction that completed; the live PC normally describes the next one. Always qualify registered trace fields with retire_valid.

06 / DESIGN AND VERIFICATION

RTL module map

The source files are small enough to study one at a time. Integration lives in rv32i_core.v.

File	Responsibility
rv32i_pc.v	Enabled, synchronous PC register; parameterized reset address.
rv32i_regfile.v	32 x 32-bit GPRs; two reads, debug read, one edge-timed write; x0 protection.
rv32i_immediate.v	I/S/B/U/J extraction and sign extension.
rv32i_decoder.v	Opcode/funct3/funct7 validation; shared control encodings; illegal-instruction default.
rv32i_alu.v	Ten complete combinational operations and a continuously assigned zero flag.
rv32i_branch.v	Six direct signed/unsigned comparisons.
rv32i_core.v	Operand and writeback MUXes, byte lanes, next-PC logic, faults and retirement trace.
rv32i_memory.v	Independent asynchronous instruction/data reads; masked stores; bounds checking.
rv32i_system.v	Convenience core + memory wrapper for simulation.

One source of control encodings

rv32i_defs.vh defines every internal ALU, immediate, branch and MUX code. Both decoding and datapath logic include it, preventing the original decoder/ALU mismatch.

Top modules

tb_system is the full-processor simulation top. rv32i_core is the synthesis top, with external memory interfaces. Synthesizing a fixed demo ROM together with the CPU can let tools remove logic that the program never uses.

07 / DESIGN AND VERIFICATION

Decode and immediate generation

The decoder selects a behavior only when the relevant instruction fields are legal.

Format	Immediate reconstructed by RTL
I	Sign-extend instruction[31:20].
S	Sign-extend {instruction[31:25], instruction[11:7]}.
B	{sign, bit 7, bits 30:25, bits 11:8, 0}, then sign-extend.
U	instruction[31:12] followed by 12 zero bits.
J	{sign, bits 19:12, bit 20, bits 30:21, 0}, then sign-extend.

Control examples

Instruction	ALU inputs	Writeback	Other controls
ADD	rs1, rs2	ALU	Register write
ADDI	rs1, I imm	ALU	Register write
LW	rs1, I imm	Load	Memory read; size=word
SW	rs1, S imm	None	Memory write; size=word
LUI	0, U imm	ALU	Register write
AUIPC	PC, U imm	ALU	Register write
JAL	Unused	PC+4	PC + J immediate
JALR	rs1, I imm	PC+4	Masked ALU target

Negative ADDI values must still select ADD: instruction bit 30 belongs to the immediate and is not a SUB selector. Shift-immediate instructions instead validate their high encoding bits. Unsupported funct7 values, widths and opcodes are illegal.

08 / DESIGN AND VERIFICATION

ALU and branch decisions

All combinational blocks assign every output. Signed and unsigned comparisons are intentional choices.

Operation	Code	Meaning
ADD / SUB	0 / 1	32-bit addition/subtraction; overflow wraps modulo 2^32.
AND / OR / XOR	2 / 3 / 4	Bitwise operations.
SLL / SRL / SRA	5 / 6 / 7	Shift by b[4:0]; SRA treats a as signed.
SLT / SLTU	8 / 9	Result is 1 or 0 using signed/unsigned ordering.

A complete combinational ALU

```
always @* begin
    case (operation)
        `ALU_ADD: result = a + b;
        `ALU_SRA: result = $signed(a) >>> b[4:0];
        // Other operations are in rv32i_alu.v.
        default:  result = 32'b0;
    endcase
end
assign zero = (result == 32'b0);
```

The continuous zero assignment avoids retaining a previous flag value. Invalid ALU codes produce zero; the instruction decoder separately rejects invalid instructions.

Branches do not rely on subtraction's sign

The branch unit compares rs1_data and rs2_data directly. For example, INT_MIN is less than INT_MAX as signed integers, even though their 32-bit difference can wrap and have the wrong sign for a naive subtraction-based comparison.

Comparison	Signed interpretation	Unsigned interpretation
0xFFFFFFFF vs 0x00000001	-1 < 1: true	4,294,967,295 < 1: false
Branch operations	BLT / BGE	BLTU / BGEU

09 / DESIGN AND VERIFICATION

Byte and halfword memory operations

The ALU produces a byte address. The memory returns an aligned word; the core selects and extends the requested lane.

Example word at address 0x200

Store 0x80FF7F01. Little-endian bytes at 0x200..0x203 are 01, 7F, FF, 80.

Load	Value written to rd	Why
LB at 0x203	0xFFFFFF80	Sign-extend byte 0x80.
LBU at 0x203	0x00000080	Zero-extend byte 0x80.
LH at 0x202	0xFFFF80FF	Sign-extend halfword 0x80FF.
LHU at 0x202	0x000080FF	Zero-extend halfword 0x80FF.
LW at 0x200	0x80FF7F01	Full aligned word.

Stores preserve neighboring bytes

Store	Strobes [3:0]	Bus operation
SB, address offset 0	0001	Write only lane 0.
SB, address offset 3	1000	Shift low byte left by 24 bits.
SH, address offset 2	1100	Shift low halfword left by 16 bits.
SW, aligned	1111	Write all four bytes.

```
shifted_load = dmem_read_data >> (8 * address[1:0]);
store_data   = rs2_data       << (8 * address[1:0]);
```

Halfwords require two-byte alignment; words require four-byte alignment. Bounds checking uses the requested access width and does not wrap a large address into the RAM array. Faults force store strobes to zero.

10 / DESIGN AND VERIFICATION

Faults, halt and reset

A faulting instruction must not partly modify architectural state.

Cause	Event	Recorded value
0	Instruction address misaligned	PC or taken target
1	Instruction access fault	Fetch address
2	Illegal instruction	Instruction word
3	EBREAK	0
4 / 5	Load misalignment / access fault	Effective address
6 / 7	Store misalignment / access fault	Effective address
11	ECALL	0

Atomic stop behavior

Fault detection disables PC update, GPR write and all store lanes. On the edge, halted becomes 1 and cause/PC/value are recorded. No retire event is produced for the faulting instruction. The state stays stable until synchronous reset.

A load to x0 still performs memory access checks. An inaccessible aligned jump target faults on the next fetch; a misaligned target faults at the branch/jump itself. An untaken branch does not inspect its unused target.

Reset contract

Active-high synchronous reset sets PC to RESET_PC, clears every GPR and clears halt/trap/trace state. It takes effect at a rising edge. The data memory has no reset input and retains its contents; testbenches clear it explicitly between cases.

Execution-environment limit

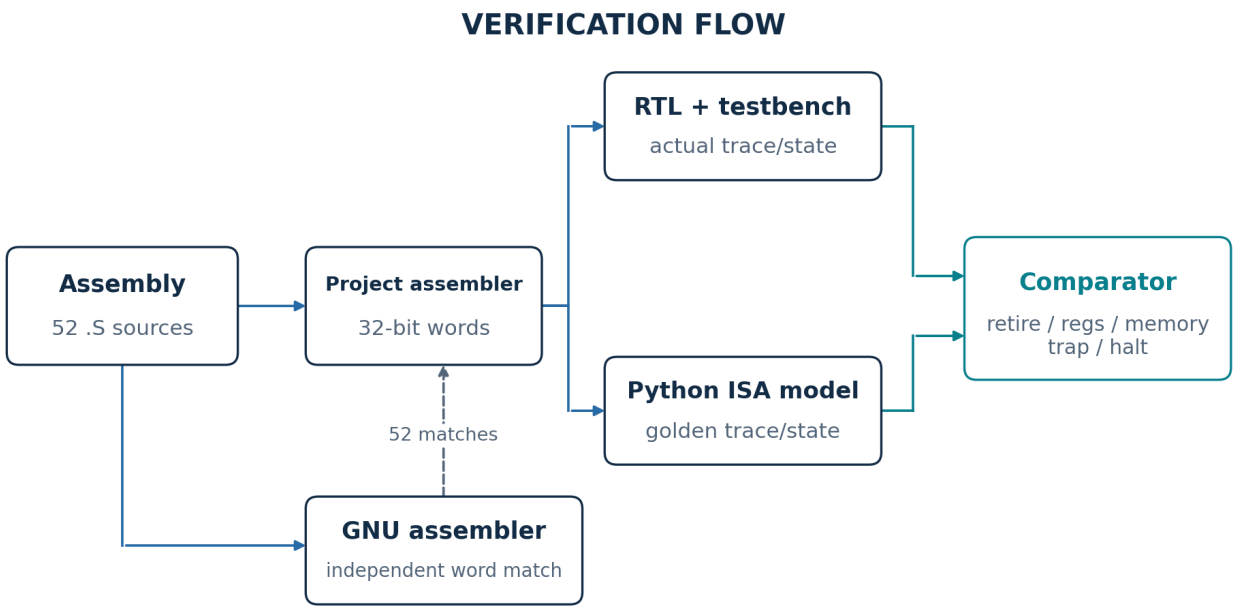
The numeric causes follow common RISC-V conventions, but they are debug outputs rather than CSRs. Cause 11 is used for ECALL; there is no implemented machine-mode state, trap vector, handler or MRET.

Fault priority is defined in docs/Interface_Contract.md and enforced by both the core and reference model.

11 / DESIGN AND VERIFICATION

Verification strategy

Waveforms help explain behavior; self-checking tests establish whether the expected result was produced.



Three levels of checks

1. Component benches validate arithmetic, immediates, decode legality, register timing and memory lanes.

2. Processor regression compares every successful instruction event with a Python architectural oracle, then checks all 32 registers and all 4096 memory bytes.

3. Trap metadata, fault-side-effect suppression, halted-state stability and reset are checked separately.

What an event contains

PC, instruction, next PC, GPR write flag/index/value, load flag, memory address, store strobes and shifted store data. Records use ten 32-bit fields; stores are also validated through final byte-addressed memory.

Independent checks and their limits

The Python interpreter does not import RTL control codes and updates individual memory bytes. GNU binutils independently matched all program encodings. Yosys checked the core structurally and rejected inferred latches.

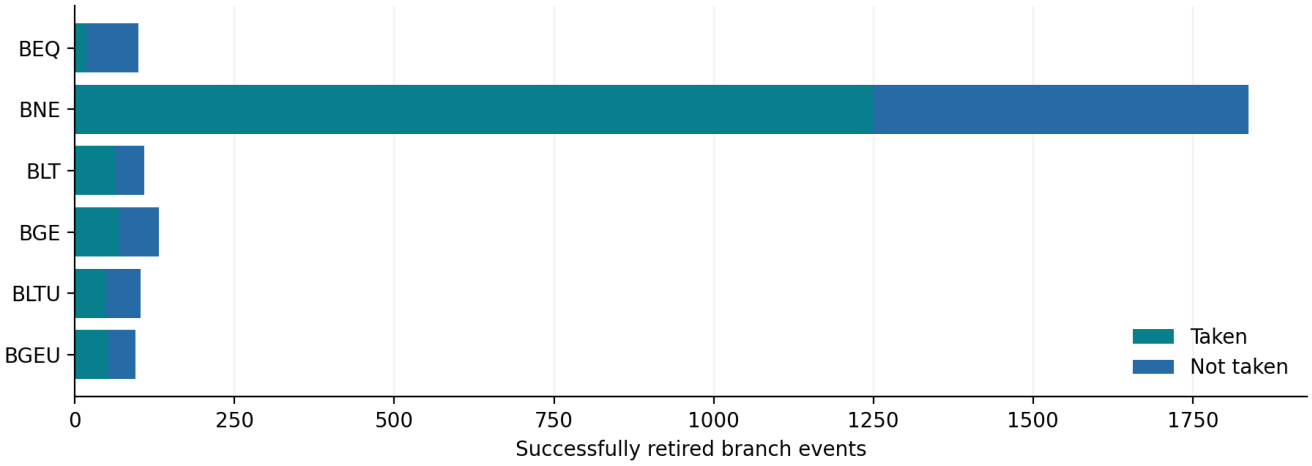
The oracle is custom project software. This regression is not a formal proof, official compliance suite or exhaustive test of every legal program.

12 / DESIGN AND VERIFICATION

Measured verification results

The numbers below come from included logs and JSON summaries, not planned runs.

Check	Actual result
Icarus Verilog	Version 12.0; all six testbenches PASS.
ALU / branch	13,752 checks.
Immediate / register file	8,192 / 98 checks.
Decoder / memory	2,093 / 13 checks.
Processor regression	52 programs; 12,604 successful events.
Random generation	16 fixed seeds; 400 generation blocks per program.
Final state / traps	All GPRs, full 4 KiB data RAM and trap metadata per case.
Mnemonic / branch bins	40/40 exercised; 12/12 branch outcomes.
GNU word comparison	52/52 exact image matches.
Yosys generic synthesis	PASS; zero inferred latches; no structural errors.



The chart counts successfully retired branches. Faulting branches and ECALL/EBREAK are checked through trap tests. Functional bins are not code/toggle coverage. See results/verified/ for original logs, CSV, VCD, source hashes and summaries.

13 / DESIGN AND VERIFICATION

Programs you can demonstrate

The examples exercise dependencies, loops and memory updates, not only isolated ALU operations.

Program	Behavior	Retired
directed_isa	All instruction families, signed corners, byte lanes, all branch outcomes and jump links.	94
fibonacci	Write F0..F11 at byte addresses 128..175.	88
bubble_sort	Sort nine signed integers at addresses 256..291.	315
random_00..15	Seeded instruction blocks with bounded loops, branches, subword memory and jumps.	12,089 total

Fibonacci output

```
Address 0x080 onwards, one 32-bit word each:
0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89
```

Signed bubble sort

```
Input : 23, -7, 0, 99, -100, 23, 1, 2147483647, -2147483648
Output: -2147483648, -100, -7, 0, 1, 23, 23, 99, 2147483647
```

Bubble sort stores a completion flag of 1 at data address 0. Successful examples stop on EBREAK; this is the expected execution-environment stop, not a test failure.

Fault and boundary tests

33 additional programs cover illegal encodings, unsupported CSR/FENCE.I, ECALL, instruction/data access faults, alignment faults, load-to-x0 behavior and taken versus untaken misaligned branch targets.

Adding a case

Add a program in tools/build_programs.py, regenerate vectors and rerun the regression. Keep bounded execution and use EBREAK to finish normal tests. Inspect mismatches rather than changing golden data to match incorrect RTL.

14 / DESIGN AND VERIFICATION

Run it in Vivado

The supplied vectors are committed, so the Vivado regression does not require Python.

GUI: first successful run

Extract the whole ZIP to a writable, short path. Open a new Vivado GUI session. Replace the example path below with your extracted folder.

```
source {C:/RISC_V/RV32I_Single_Cycle_Project/scripts/vivado/create_project.tcl}
launch_simulation
run all
```

The default part is xc7z020clg400-1. Set RV32I_PART to another installed part before sourcing the script if needed.

Required pass messages

```
SYSTEM_SUMMARY cases=52 retired=12604
TESTBENCH_PASS tb_system
```

A failure stops with $fatal. The PASS file and retire CSV appear in results/user_runs/xsim/. Batch mode deletes old pass markers before each testbench.

All six testbenches

```
vivado -mode batch -source scripts/vivado/run_regression.tcl
# Require: VIVADO_REGRESSION_PASS
```

Waveform study

Expand tb_system/dut/u_core. Start with pc, instruction, rs1_data, rs2_data, immediate, alu_a, alu_b, alu_result and writeback_data. Add register_write_enable, next_pc and branch_taken; then memory signals and registered trace fields.

Use hexadecimal for instructions/addresses/data. After a rising edge, live PC normally points to the next instruction, while trace_pc identifies the completed one.

Vivado/XSim was not executed during preparation. These scripts are reproduction entry points; keep your local logs and screenshots. Detailed guidance and troubleshooting are in docs/Vivado_Guide.md.

15 / DESIGN AND VERIFICATION

Synthesis and engineering choices

A frequency constraint is a target; a measured implementation needs its own evidence.

Run core-only Vivado synthesis

```
vivado -mode batch -source scripts/vivado/run_synthesis.tcl
```

The script emits utilization, timing-summary and methodology reports plus a synthesized checkpoint in results/user_runs/vivado_synthesis/. The top is rv32i_core with external memory ports; debug and retirement circuitry contribute to area.

Actual generic synthesis result

Yosys generic synthesis completed with 7,981 mapped logic cells and zero inferred latches. This cell count is not an FPGA LUT/FF utilization count. It includes the register file and observation logic; simulation memories are outside the synthesis top.

The 10 ns constraint

core_clock.xdc specifies a 10 ns target. No external-memory I/O delays, board pins, placement, routing or bitstream are supplied. Synthesis estimates alone cannot establish board timing closure or a maximum operating frequency.

Corrections relative to the learning reference

Issue identified	Implementation in this package
ALU decoder/datapath mismatch	Shared definitions; all operations tested.
Incomplete zero-flag assignments	Continuous zero assignment; complete combinational defaults.
Clocked read in a single-cycle load path	Combinational memory-read contract.
Ambiguous PC+4 writeback diagram	Explicit third writeback input for JAL/JALR.
Limited / inconsistent testbench hookup	Six self-checking benches plus architectural trace comparison.

Original repository commit inspected: 577eb7e10d091069c6b1e67968ad49eca284749c. The new package does not redistribute its RTL or report.

16 / DESIGN AND VERIFICATION

Review, resume and next steps

Use the package as a learning project, then collect evidence from your own runs.

Before putting it on your resume

Explain ADD, LW, SW and JALR through the block diagram. Reproduce all Vivado tests. Make one controlled RTL change, predict the effect and demonstrate that the checker catches the resulting mismatch. Keep your new logs and screenshots.

Suggested project description

Implemented a modular 32-bit single-cycle RISC-V core with 37 RV32I datapath instructions, subword memory operations and fault reporting; developed six self-checking testbenches and checked 52 programs / 12,604 instruction events against an architectural reference flow.

Use this wording only after reviewing and reproducing the work. Add Vivado metrics only after local runs. The project is not UVM, formally verified, OS-capable or board-demonstrated.

Expand it in stages

Stage	Next engineering task
1	Reproduce XSim and core synthesis; explain the measured results.
2	Add independent Spike/Sail or architectural-test checks.
3	Introduce a memory handshake and synchronous block RAM.
4	Pipeline the core; add forwarding, stalls and branch flush tests.
5	Build a small system with peripherals and documented CSR/trap support.

References and attribution

RISC-V International - RV32I Base Integer Instruction Set, version 2.1

AMD - UG900, Vivado Design Suite User Guide: Logic Simulation (2022.1)

AMD - UG835, launch_simulation Tcl command

Learning reference - Samad Abbasi, RISC-V-Single-Cycle-Processor

Supporting evidence and commands are included with the source. The implementation and tests are newly written for this package.